



DEPARTAMENTO DE INGENIERÍA MATEMÁTICA
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
UNIVERSIDAD DE CHILE
MA4703 CONTROL ÓPTIMO

INFORME DE PROYECTO

DEEP LEARNING COMO UN PROBLEMA DE CONTROL ÓPTIMO

INTEGRANTES:	DIEGO ACUÑA S. MAXIMILIANO VALLADARES G. NICOLÁS FUENZALIDA S.
PROFESOR:	HÉCTOR RAMÍREZ C.
AUXILIAR:	DIEGO OLGUÍN W.
AYUDANTES:	CARLOS ANTIL C. LUIS FUENTES C.

FECHA DE ENTREGA: 4 DE DICIEMBRE DE 2024
SANTIAGO DE CHILE

Índice de Contenidos

1. Introducción	1
1.1. Conceptos Fundamentales del Deep Learning y la Arquitectura ResNet . . .	2
1.1.1. Redes Neuronales Artificiales	2
1.1.2. Entrenamiento de Redes Neuronales	3
1.1.3. Deep Learning y Desafíos de Profundidad	3
1.1.4. Arquitectura ResNet	3
1.1.5. Interpretación como Sistema Dinámico Controlado	4
1.2. Formulación Matemática del Problema de Control Óptimo	5
1.2.1. Formulación del Problema	5
1.2.2. Principio del Máximo de Pontryaguin	6
1.2.3. Ecuación de Hamilton-Jacobi-Bellman	9
1.2.4. Análisis Comparativo entre PMP y HJB	10
1.2.5. Aplicación al Problema de Clasificación	10
2. Planteamiento y Metodología	12
2.1. Discretización Temporal y Formulación del Problema	12
2.2. Formulación del Pre-Hamiltoniano y Ecuaciones Adjuntas	13
2.3. Condiciones de Optimalidad	13
2.4. Implementación Numérica	13
2.4.1. Función de Activación y su Derivada	14
2.4.2. Generación de Datos Sintéticos	14
2.4.3. Formulación del Problema de Optimización	14
2.4.4. Resolución del Problema	15
2.5. Algoritmo Implementado	15
2.6. Detalles Técnicos y Consideraciones	16
2.6.1. Uso de Aproximaciones Suaves	16
2.6.2. Dimensionalidad y Eficiencia Computacional	16
2.6.3. Validación de Resultados	16
3. Resultados	17
4. Conclusiones	19
Referencias	21
Anexo A. Código	22

Índice de Figuras

1. Transformación de los datos a medida que aumenta la profundidad de la red . . . 17

Índice de Códigos

- A.1. Formulación del problema 22
A.2. Visualización de resultados 25

1. Introducción

La clasificación de datos es una de las tareas centrales en el campo del aprendizaje de máquinas y la inteligencia artificial. Consiste en asignar a cada elemento de un conjunto de datos una etiqueta o clase entre un conjunto finito y predefinido de posibles categorías. Formalmente, se busca una función $g : \mathbb{R}^n \rightarrow \{c_0, c_1, \dots, c_{K-1}\}$ que asigne correctamente cada vector de características $x \in \mathbb{R}^n$ a una de las K clases disponibles. En el caso particular de la clasificación binaria, $K = 2$, y el problema se reduce a distinguir entre dos posibles categorías.

En el aprendizaje supervisado, este problema se aborda típicamente mediante la estimación de los parámetros óptimos de una función parametrizada utilizando un conjunto de datos de entrenamiento $\{(x_i, c_i)\}_{i=1}^m$, donde $c_i \in \{c_0, c_1\}$. El objetivo es minimizar una función de costo que cuantifica la discrepancia entre las predicciones del modelo y las etiquetas reales de los datos de entrenamiento.

Recientemente, se ha establecido una conexión profunda entre el deep learning y los problemas de control óptimo, permitiendo analizar y diseñar arquitecturas de redes neuronales desde esta perspectiva matemática. En particular, ciertas arquitecturas de redes neuronales profundas pueden interpretarse como discretizaciones en el tiempo de sistemas dinámicos controlados, donde los parámetros de la red (pesos y sesgos) actúan como controles que influyen en la evolución de los estados internos de la red.

Este proyecto se enfoca en estudiar esta interpretación para problemas de clasificación, utilizando la arquitectura de Redes Neuronales Residuales (ResNet) y aplicando las herramientas teóricas y numéricas del control óptimo. Se pretende formular el problema de entrenamiento de una red neuronal como un problema de control óptimo, analizarlo matemáticamente mediante el Principio del Máximo de Pontryaguin y las ecuaciones de Hamilton-Jacobi-Bellman, y desarrollar algoritmos numéricos para su resolución.

A continuación, se introducirán en detalle los conceptos fundamentales del deep learning y, en particular, de la arquitectura ResNet, proporcionando el fundamento matemático necesario para el análisis posterior.

1.1. Conceptos Fundamentales del Deep Learning y la Arquitectura ResNet

1.1.1. Redes Neuronales Artificiales

Una red neuronal artificial (RNA) es un modelo computacional inspirado en la estructura y funcionamiento del cerebro humano. Está compuesta por unidades elementales llamadas neuronas artificiales, organizadas en capas y conectadas entre sí mediante enlaces ponderados. Cada neurona recibe entradas, realiza una combinación lineal de estas entradas y aplica una función de activación no lineal para producir una salida.

Matemáticamente, una RNA de L capas puede describirse mediante las siguientes ecuaciones:

$$\begin{aligned} z^{[0]} &= x, \\ a^{[l]} &= W^{[l]}z^{[l-1]} + b^{[l]}, \quad l = 1, \dots, L, \\ z^{[l]} &= \sigma^{[l]}(a^{[l]}), \quad l = 1, \dots, L, \end{aligned} \tag{1}$$

donde:

- $x \in \mathbb{R}^n$ es el vector de entrada,
- $z^{[l]} \in \mathbb{R}^{n_l}$ es el vector de activaciones de la capa l ,
- $W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$ es la matriz de pesos entre las capas $l - 1$ y l ,
- $b^{[l]} \in \mathbb{R}^{n_l}$ es el vector de sesgos de la capa l ,
- $\sigma^{[l]} : \mathbb{R}^{n_l} \rightarrow \mathbb{R}^{n_l}$ es la función de activación aplicada componente a componente en la capa l .

La función de activación σ introduce no linealidad en el modelo, permitiendo que la red aprenda relaciones complejas y no lineales entre las variables de entrada y salida. Ejemplos comunes de funciones de activación incluyen:

- Función sigmoide: $\sigma(x) = \frac{1}{1+e^{-x}}$,
- Tangente hiperbólica: $\sigma(x) = \tanh(x)$,
- Unidad Lineal Rectificada (ReLU): $\sigma(x) = \max(0, x)$.

1.1.2. Entrenamiento de Redes Neuronales

El proceso de entrenamiento de una RNA implica ajustar los parámetros $\{W^{[l]}, b^{[l]}\}$ para minimizar una función de costo J que mide el error entre las predicciones de la red y las etiquetas reales en el conjunto de entrenamiento. Comúnmente, para problemas de clasificación, se utiliza la función de costo de entropía cruzada o el error cuadrático medio.

El entrenamiento se realiza mediante algoritmos de optimización basados en el descenso de gradiente. La técnica de retropropagación (backpropagation) se emplea para calcular eficientemente el gradiente de la función de costo respecto a los parámetros de la red, aplicando la regla de la cadena a través de las capas.

1.1.3. Deep Learning y Desafíos de Profundidad

Las redes neuronales profundas (deep neural networks) son aquellas que constan de múltiples capas ocultas, lo que les permite aprender representaciones jerárquicas y características de alto nivel de los datos. Sin embargo, incrementar la profundidad de las redes plantea desafíos significativos:

- **Desaparición o Explosión del Gradiente:** A medida que las señales se propagan hacia atrás durante el entrenamiento, los gradientes pueden disminuir o aumentar exponencialmente, dificultando la actualización efectiva de los pesos en capas profundas.
- **Problemas de Generalización:** Redes muy profundas pueden ser propensas al sobreajuste, capturando ruido en los datos de entrenamiento y disminuyendo su capacidad de generalización a nuevos datos.

1.1.4. Arquitectura ResNet

Para superar estos desafíos, He et al. introdujeron en 2016 la arquitectura de Redes Neuronales Residuales (ResNet) [1], que permite entrenar redes significativamente más profundas sin degradación en el rendimiento. La clave de ResNet es la introducción de conexiones residuales o *skip connections*, que facilitan el flujo de información y gradientes a través de la red.

El bloque residual básico de una ResNet se define mediante la siguiente ecuación:

$$y^{[j+1]} = y^{[j]} + \mathcal{F}(y^{[j]}, u^{[j]}), \quad (2)$$

donde:

- $y^{[j]} \in \mathbb{R}^n$ es el vector de activaciones en la capa j ,
- $\mathcal{F}(y^{[j]}, u^{[j]})$ representa la función residual aprendida, parametrizada por $u^{[j]}$,

- $u^{[j]} = \{W^{[j]}, b^{[j]}\}$ son los parámetros (pesos y sesgos) asociados a la capa j .

La función residual $\mathcal{F}$ generalmente consiste en una secuencia de operaciones lineales y no lineales, por ejemplo:

$$\mathcal{F}(y^{[j]}, u^{[j]}) = \sigma(W^{[j]}y^{[j]} + b^{[j]}), \quad (3)$$

donde σ es una función de activación.

La ecuación (2) puede interpretarse como la discretización de Euler explícita de una ecuación diferencial ordinaria (EDO):

$$\frac{dy(t)}{dt} = f(y(t), u(t)), \quad (4)$$

donde:

- $y(t)$ es el estado del sistema en el tiempo continuo t ,
- $f(y, u) = \mathcal{F}(y, u)$.

Al considerar $\Delta t = 1$ y discretizar el tiempo en pasos $t_j = j$, obtenemos $y^{[j+1]} = y^{[j]} + f(y^{[j]}, u^{[j]})$, que coincide con la estructura del bloque residual.

1.1.5. Interpretación como Sistema Dinámico Controlado

La representación continua permite interpretar el entrenamiento de una ResNet como un problema de control óptimo sobre un sistema dinámico controlado. En este marco, el estado del sistema es el vector de activaciones $y(t)$, y los controles $u(t)$ corresponden a los parámetros de la red que influyen en la evolución del estado.

El objetivo es encontrar los controles $u(t)$ que minimicen un funcional de costo J , típicamente asociado al error de predicción en el tiempo final T :

$$J(u) = \frac{1}{2} \sum_{i=1}^m |C(Wy_i(T) + \mu) - c_i|^2, \quad (5)$$

sujeto a la dinámica del sistema para cada dato de entrenamiento x_i :

$$\frac{dy_i(t)}{dt} = f(y_i(t), u(t)), \quad y_i(0) = x_i, \quad (6)$$

donde:

- $y_i(t)$ es el estado del sistema correspondiente al dato x_i ,
- W y μ son parámetros de la capa de salida,

- C es la función de hipótesis que mapea la salida continua al conjunto discreto de clases,
- c_i es la etiqueta real del dato x_i .

La formulación (5) y (6) constituye un problema de control óptimo, donde buscamos la función de control $u(t)$ que minimiza el costo total $J(u)$ al final del horizonte temporal T .

Esta interpretación proporciona varias ventajas:

- **Fundamento Teórico Sólido:** Permite aplicar la teoría matemática bien establecida del control óptimo para analizar la existencia, unicidad y propiedades de las soluciones.
- **Algoritmos Numéricos Eficientes:** Las técnicas numéricas desarrolladas en control óptimo, como métodos indirectos basados en el Principio del Máximo de Pontryaguin o métodos directos de optimización, pueden adaptarse para entrenar redes neuronales.
- **Análisis de Estabilidad y Robustez:** El marco de sistemas dinámicos facilita el estudio de la estabilidad de la red y su comportamiento ante perturbaciones, mejorando su robustez.

En la siguiente sección, se explorará en detalle las herramientas matemáticas del control óptimo aplicadas a este problema, incluyendo el Principio del Máximo de Pontryaguin y las ecuaciones de Hamilton-Jacobi-Bellman.

1.2. Formulación Matemática del Problema de Control Óptimo

En esta sección, se establecerá la formulación del problema de clasificación como un problema de control óptimo. Se derivará el Principio del Máximo de Pontryaguin y las ecuaciones de Hamilton-Jacobi-Bellman, proporcionando un marco teórico para el análisis y resolución del problema.

1.2.1. Formulación del Problema

Consideremos un conjunto de datos de entrenamiento $\{(x_i, c_i)\}_{i=1}^m$, donde $x_i \in \mathbb{R}^n$ son los vectores de características y $c_i \in \{c_0, c_1\}$ son las etiquetas correspondientes.

La dinámica del sistema se modela mediante una ecuación diferencial ordinaria (EDO) controlada, que representa la evolución de los estados internos de la red neuronal a lo largo del tiempo continuo $t \in [0, T]$:

$$\frac{dy_i(t)}{dt} = f(y_i(t), u(t)), \quad y_i(0) = x_i, \quad (7)$$

donde:

- $y_i(t) \in \mathbb{R}^n$ es el estado del sistema en el tiempo t para el dato x_i .
- $u(t) = (K(t), \beta(t))$ es el control, donde:
 - $K(t) \in \mathbb{R}^{n \times n}$ es la matriz de pesos dependiente del tiempo.
 - $\beta(t) \in \mathbb{R}^n$ es el vector de sesgos dependiente del tiempo.
- $f : \mathbb{R}^n \times \mathbb{R}^{n \times n} \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ es una función continua que define la dinámica del sistema, dada por:

$$f(y_i(t), u(t)) = \sigma(K(t)y_i(t) + \beta(t)), \quad (8)$$

donde $\sigma : \mathbb{R}^n \rightarrow \mathbb{R}^n$ es una función de activación aplicada componente a componente.

El objetivo es encontrar el control óptimo $u^*(t)$ que minimice el funcional de costo:

$$J(u) = \frac{1}{2} \sum_{i=1}^m |\varphi(y_i(T)) - c_i|^2, \quad (9)$$

donde:

- $y_i(T)$ es el estado final en el tiempo T para el dato x_i .
- $\varphi : \mathbb{R}^n \rightarrow \mathbb{R}$ es una función de salida, definida como:

$$\varphi(y_i(T)) = C(Wy_i(T) + \mu), \quad (10)$$

con:

- $W \in \mathbb{R}^{1 \times n}$ es el vector de pesos de la capa de salida.
- $\mu \in \mathbb{R}$ es el sesgo de la capa de salida.
- $C : \mathbb{R} \rightarrow \mathbb{R}$ es una función de activación, por ejemplo, la función sigmoide.

1.2.2. Principio del Máximo de Pontryaguin

El Principio del Máximo de Pontryaguin proporciona condiciones necesarias para la optimalidad en problemas de control óptimo. Establece que, si $u^*(t)$ es un control óptimo y $y_i^*(t)$ es la trayectoria de estado correspondiente, entonces existe una función adjunta $p_i(t) \in \mathbb{R}^n$ no trivial tal que:

1. Ecuación del estado:

$$\frac{dy_i^*(t)}{dt} = f(y_i^*(t), u^*(t)), \quad y_i^*(0) = x_i. \quad (11)$$

2. Ecuación del adjunto:

$$\frac{dp_i(t)}{dt} = -\frac{\partial H_i}{\partial y_i}, \quad p_i(T) = \frac{\partial \Phi_i}{\partial y_i(T)}, \quad (12)$$

donde H_i es el Hamiltoniano y $\Phi_i(y_i(T))$ es el costo terminal para el dato x_i .

3. Condición de optimalidad:

$$H_i(y_i^*(t), p_i(t), u^*(t)) = \min_{u \in \mathcal{U}} H_i(y_i^*(t), p_i(t), u), \quad (13)$$

donde $\mathcal{U}$ es el conjunto de controles admisibles.

Definición del Hamiltoniano

Para cada dato x_i , el Hamiltoniano $H_i : \mathbb{R}^n \times \mathbb{R}^n \times \mathcal{U} \rightarrow \mathbb{R}$ se define como:

$$H_i(y_i, p_i, u) = p_i^\top f(y_i, u). \quad (14)$$

Dado que el costo es nulo (el costo se acumula solo en $t = T$), no aparece un término adicional en el Hamiltoniano.

Cálculo de las Derivadas Necesarias

El costo terminal para el dato x_i es:

$$\Phi_i(y_i(T)) = \frac{1}{2} |\varphi(y_i(T)) - c_i|^2. \quad (15)$$

Por lo tanto, la derivada respecto al estado final es:

$$p_i(T) = \frac{\partial \Phi_i}{\partial y_i(T)} = (\varphi(y_i(T)) - c_i) \frac{\partial \varphi}{\partial y_i(T)}. \quad (16)$$

Calculamos la derivada parcial:

$$\frac{\partial H_i}{\partial y_i} = \left(\frac{\partial f}{\partial y_i} \right)^\top p_i, \quad (17)$$

donde la derivada de f respecto a y_i es:

$$\frac{\partial f}{\partial y_i} = \frac{\partial \sigma(Ky_i + \beta)}{\partial y_i} = K^\top \text{diag}(\sigma'(Ky_i + \beta)), \quad (18)$$

siendo $\text{diag}(\sigma')$ una matriz diagonal con las derivadas de σ evaluadas en cada componente.

Condición de Optimalidad

La condición de optimalidad se expresa como:

$$u^*(t) = \arg \min_{u \in \mathcal{U}} H_i(y_i^*(t), p_i(t), u). \quad (19)$$

Sustituyendo el Hamiltoniano:

$$H_i(y_i^*(t), p_i(t), u) = p_i(t)^\top \sigma(K y_i^*(t) + \beta). \quad (20)$$

Para encontrar el mínimo respecto a $u = (K, \beta)$, calculamos las derivadas parciales del Hamiltoniano respecto a K y β :

$$\frac{\partial H_i}{\partial K} = p_i(t)^\top \text{diag}(\sigma'(K y_i^*(t) + \beta)) y_i^{*\top}(t), \quad (21)$$

$$\frac{\partial H_i}{\partial \beta} = p_i(t)^\top \text{diag}(\sigma'(K y_i^*(t) + \beta)). \quad (22)$$

Igualando a cero para encontrar el mínimo:

$$\frac{\partial H_i}{\partial K} = 0, \quad (23)$$

$$\frac{\partial H_i}{\partial \beta} = 0. \quad (24)$$

Resumen de las Ecuaciones del PMP

Las ecuaciones que deben resolverse para cada dato x_i son:

- **Ecuación del estado:**

$$\frac{dy_i^*(t)}{dt} = \sigma(K^*(t)y_i^*(t) + \beta^*(t)), \quad y_i^*(0) = x_i. \quad (25)$$

- **Ecuación del adjunto:**

$$\frac{dp_i(t)}{dt} = -K^{*\top}(t) \text{diag}(\sigma'(K^*(t)y_i^*(t) + \beta^*(t))) p_i(t), \quad p_i(T) = (\varphi(y_i^*(T)) - c_i) \frac{\partial \varphi}{\partial y_i(T)}. \quad (26)$$

- **Condiciones de optimalidad:**

$$p_i(t)^\top \text{diag}(\sigma'(K^*(t)y_i^*(t) + \beta^*(t))) y_i^{*\top}(t) = 0, \quad (27)$$

$$p_i(t)^\top \text{diag}(\sigma'(K^*(t)y_i^*(t) + \beta^*(t))) = 0. \quad (28)$$

Estas ecuaciones constituyen un sistema acoplado de ecuaciones diferenciales y algebraicas que deben resolverse simultáneamente para obtener el control óptimo $u^*(t)$ y las trayectorias

de estado $y_i^*(t)$.

1.2.3. Ecuación de Hamilton-Jacobi-Bellman

La ecuación de Hamilton-Jacobi-Bellman (HJB) es una ecuación diferencial parcial que proporciona una condición necesaria y suficiente para la optimalidad. La función valor $V(y, t)$ representa el costo mínimo acumulado desde el estado y en el tiempo t hasta el tiempo final T :

$$V(y, t) = \min_{u(\cdot)} \int_t^T L(y(s), u(s)) ds + \Phi(y(T)), \quad (29)$$

donde:

- $L(y, u)$ es el costo, que en nuestro caso es cero ($L(y, u) = 0$) ya que el costo solo se acumula en el tiempo final.
- $\Phi(y(T))$ es el costo terminal definido previamente.

La ecuación HJB se expresa como:

$$-\frac{\partial V}{\partial t}(y, t) = \min_{u \in \mathcal{U}} \{H(y, \nabla_y V(y, t), u)\}, \quad (30)$$

con la condición terminal:

$$V(y, T) = \Phi(y). \quad (31)$$

Derivación de la Ecuación HJB

Dado que el costo es cero, la ecuación HJB se simplifica a:

$$-\frac{\partial V}{\partial t}(y, t) = \min_{u \in \mathcal{U}} \left\{ \nabla_y V(y, t)^\top f(y, u) \right\}. \quad (32)$$

El Hamiltoniano para este problema es:

$$H(y, p, u) = p^\top f(y, u), \quad (33)$$

donde $p = \nabla_y V(y, t)$ es el gradiente espacial de la función valor.

Condición de Optimalidad de la HJB

La minimización puntual del Hamiltoniano implica que el control óptimo $u^*(y, t)$ satisface:

$$u^*(y, t) = \arg \min_{u \in \mathcal{U}} \left\{ \nabla_y V(y, t)^\top \sigma(Ky + \beta) \right\}. \quad (34)$$

Esta minimización se realiza para cada estado y y tiempo t , y determina el control óptimo en términos de la función valor.

Resolución de la Ecuación HJB

Resolver la ecuación HJB directamente es una tarea altamente compleja debido a:

- La no linealidad introducida por la función de activación σ .
- La alta dimensionalidad del espacio de estados $\mathbb{R}^n$.
- La dependencia del control óptimo en el gradiente de la función valor $\nabla_y V(y, t)$.

Por lo tanto, se suelen emplear métodos numéricos y aproximaciones, como:

- Métodos basados en esquemas de diferencias finitas para discretizar el espacio y el tiempo.
- Métodos de programación dinámica aproximada.
- Técnicas de aprendizaje por refuerzo que aproximan la función valor mediante funciones de aproximación parametrizadas.

1.2.4. Análisis Comparativo entre PMP y HJB

El Principio del Máximo de Pontryaguin y la ecuación de Hamilton-Jacobi-Bellman son herramientas complementarias en el análisis de problemas de control óptimo:

- El PMP proporciona condiciones necesarias para la optimalidad en términos de ecuaciones diferenciales ordinarias para el estado y el adjunto, junto con condiciones de optimalidad que deben satisfacerse a lo largo de las trayectorias óptimas.
- La HJB ofrece una condición necesaria y suficiente para la optimalidad, formulada como una ecuación diferencial parcial sobre la función valor $V(y, t)$.

En la práctica, el PMP es más adecuado para problemas de dimensión alta, ya que implica resolver un sistema de EDOs a lo largo de una trayectoria específica. La HJB, aunque proporciona información global sobre el problema, es computacionalmente intratable en espacios de alta dimensión debido al fenómeno conocido como la "maldición de la dimensionalidad".

1.2.5. Aplicación al Problema de Clasificación

La formulación del entrenamiento de redes neuronales profundas como un problema de control óptimo permite:

- Interpretar los parámetros de la red (pesos y sesgos) como controles que influyen la dinámica del estado interno.
- Aplicar técnicas del control óptimo para encontrar los parámetros que minimizan el error de clasificación.

- Analizar la estabilidad y robustez de las soluciones óptimas desde una perspectiva teórica.

En particular, el uso del PMP facilita el diseño de algoritmos numéricos que iterativamente actualizan los controles $u(t)$ basándose en las ecuaciones de estado y adjunto, análogamente a como se realiza la retropropagación del gradiente en el entrenamiento de redes neuronales.

Recapitulando, la derivación del Principio del Máximo de Pontryaguin y las ecuaciones de Hamilton-Jacobi-Bellman para el problema de clasificación nos proporciona un marco matemático sólido para abordar el entrenamiento de redes neuronales profundas como problemas de control óptimo. Esto no solo permite una comprensión más profunda de los mecanismos subyacentes al deep learning, sino que también abre la puerta a nuevas técnicas y algoritmos basados en teoría de control que podrían mejorar el rendimiento y la eficiencia de los modelos de clasificación.

En las siguientes secciones, se explorarán métodos numéricos para resolver las ecuaciones del PMP y se implementarán algoritmos basados en estas teorías para comparar su desempeño con los métodos tradicionales de deep learning.

2. Planteamiento y Metodología

En esta sección, se detallará el enfoque metodológico adoptado para resolver el problema de clasificación mediante la interpretación del entrenamiento de una red neuronal residual (ResNet) como un problema de control óptimo. Se implementará una discretización en el tiempo de las ecuaciones de estado y adjuntas, y se utilizarán técnicas de optimización numérica para encontrar los controles óptimos que minimicen la función de costo.

2.1. Discretización Temporal y Formulación del Problema

Considerando que N es el número de capas de la red neuronal, se realizará una discretización uniforme del intervalo de tiempo $[0, T]$ con un paso temporal $\Delta t = \frac{T}{N}$. Las variables de estado y control se expresarán de la siguiente manera:

$$y = (y^{[0]}, y^{[1]}, \dots, y^{[N]}), \quad \text{donde cada } y^{[j]} \in \mathbb{R}^n, \quad (35)$$

$$u = (u^{[0]}, u^{[1]}, \dots, u^{[N-1]}), \quad \text{donde cada } u^{[j]} = (K^{[j]}, \beta^{[j]}), \quad (36)$$

con $K^{[j]} \in \mathbb{R}^{n \times n}$ y $\beta^{[j]} \in \mathbb{R}^n$ representando los pesos y sesgos de la capa j -ésima de la red. La dinámica discretizada del sistema, basada en la estructura de ResNet, se define como:

$$y^{[j+1]} = y^{[j]} + \Delta t \cdot f(y^{[j]}, u^{[j]}), \quad j = 0, 1, \dots, N-1, \quad (37)$$

donde la función f es:

$$f(y^{[j]}, u^{[j]}) = \sigma(K^{[j]}y^{[j]} + \beta^{[j]}), \quad (38)$$

y $\sigma : \mathbb{R}^n \rightarrow \mathbb{R}^n$ es una función de activación aplicada componente a componente. El objetivo es minimizar la función de costo discreta:

$$J(u) = \frac{1}{2} \sum_{i=1}^m \|y_i^{[N]} - c_i\|^2, \quad (39)$$

donde $y_i^{[N]}$ es el estado final para el dato x_i y c_i es su etiqueta asociada.

2.2. Formulación del Pre-Hamiltoniano y Ecuaciones Adjuntas

Aplicando el Principio del Máximo de Pontryaguin en el contexto discreto, definimos el pre-Hamiltoniano para cada dato x_i en el instante j como:

$$H^{[j]}(y_i^{[j]}, u^{[j]}, p_i^{[j+1]}) = (p_i^{[j+1]})^\top f(y_i^{[j]}, u^{[j]}), \quad (40)$$

donde $p_i^{[j+1]} \in \mathbb{R}^n$ es la variable adjunta en el instante $j + 1$ para el dato x_i .

Las ecuaciones adjuntas discretizadas se obtienen a partir de la derivación del pre-Hamiltoniano respecto al estado, resultando en:

$$p_i^{[j]} = p_i^{[j+1]} - \Delta t \left(\frac{\partial f}{\partial y}(y_i^{[j]}, u^{[j]}) \right)^\top p_i^{[j+1]}, \quad j = N - 1, N - 2, \dots, 0, \quad (41)$$

con la condición terminal:

$$p_i^{[N]} = \frac{\partial \Phi}{\partial y}(y_i^{[N]}) = y_i^{[N]} - c_i. \quad (42)$$

Aquí, $\Phi(y_i^{[N]}) = \frac{1}{2} \|y_i^{[N]} - c_i\|^2$ es el costo terminal.

2.3. Condiciones de Optimalidad

La condición de optimalidad discreta se expresa como:

$$0 = \frac{\partial H^{[j]}}{\partial u^{[j]}} = \left(\frac{\partial f}{\partial u^{[j]}}(y_i^{[j]}, u^{[j]}) \right)^\top p_i^{[j+1]}, \quad j = 0, 1, \dots, N - 1. \quad (43)$$

Desglosando las derivadas parciales, obtenemos:

$$\frac{\partial f}{\partial K^{[j]}} = \sigma' \left(K^{[j]} y_i^{[j]} + \beta^{[j]} \right) y_i^{[j]\top}, \quad (44)$$

$$\frac{\partial f}{\partial \beta^{[j]}} = \sigma' \left(K^{[j]} y_i^{[j]} + \beta^{[j]} \right), \quad (45)$$

donde σ' es la derivada de la función de activación aplicada componente a componente.

2.4. Implementación Numérica

Para resolver el problema de control óptimo discretizado, se empleó el lenguaje de programación Julia, aprovechando su eficiencia y capacidades para cálculos numéricos y optimización. A continuación, se detallan los componentes clave de la implementación.

2.4.1. Función de Activación y su Derivada

Se utilizó la función de activación ReLU (Rectified Linear Unit) definida como:

$$\text{ReLU}(x) = \max(0, x). \quad (46)$$

La derivada de ReLU presenta una discontinuidad en $x = 0$, lo cual puede causar dificultades en los algoritmos de optimización. Para mitigar este problema, se adoptó una aproximación suave de la derivada de ReLU utilizando la función sigmoide:

$$\text{ReLU}'(x) \approx \frac{1}{1 + e^{-kx}}, \quad (47)$$

con un valor grande de k (por ejemplo, $k = 10$) para aproximar la función escalón.

2.4.2. Generación de Datos Sintéticos

Se generaron datos sintéticos para un problema de clasificación binaria:

- Se crearon n_{samples} muestras con n_{features} características, siguiendo una distribución normal estándar.
- Se normalizaron los datos para tener media cero y desviación estándar uno en cada característica.
- Se definieron pesos verdaderos aleatorios y se calcularon las etiquetas continuas añadiendo ruido gaussiano.
- Se convirtieron las etiquetas continuas en clases binarias utilizando la mediana como umbral.

2.4.3. Formulación del Problema de Optimización

Se utilizó el paquete JuMP para formular y resolver el problema de optimización. El modelo incluye:

- **Variables de decisión:**
 - Estados $y^{[j]} \in \mathbb{R}^{n_{\text{samples}} \times n_{\text{features}}}$ para $j = 0, 1, \dots, N$.
 - Controles $K^{[j]} \in \mathbb{R}^{n_{\text{features}} \times n_{\text{features}}}$ y $\beta^{[j]} \in \mathbb{R}^{n_{\text{features}}}$ para $j = 0, 1, \dots, N - 1$.
 - Variables adjuntas $p^{[j]} \in \mathbb{R}^{n_{\text{samples}} \times n_{\text{features}}}$ para $j = 0, 1, \dots, N$.
- **Restricciones:**
 - **Dinámica del estado:** Implementada según la ecuación (37) para cada muestra y cada capa.

- **Ecuaciones adjuntas:** Implementadas según la ecuación (41) en orden inverso (desde N hasta 0).
- **Condiciones iniciales y terminales:** Estados iniciales $y^{[0]} = x_i$ y condiciones terminales de los adjuntos según la ecuación (42).
- **Función objetivo:** Minimizar el costo total definido en la ecuación (39).

2.4.4. Resolución del Problema

El problema de optimización se resolvió utilizando el solver **Ipop**, que es adecuado para problemas de programación no lineal a gran escala. Se estableció una tolerancia de convergencia apropiada para garantizar la precisión de la solución.

2.5. Algoritmo Implementado

El algoritmo implementado en Julia puede resumirse en los siguientes pasos:

1. Inicialización:

- Generación de los datos sintéticos de entrada X y las etiquetas binarias y_{classes} .
- Definición de los parámetros: número de muestras, características, capas y paso temporal Δt .

2. Definición del Modelo de Optimización:

- Creación del modelo **JuMP** y configuración del solver **Ipop**.
- Declaración de las variables de estado, control y adjuntas.
- Configuración de las condiciones iniciales para los estados y terminales para los adjuntos.

3. Construcción de las Restricciones:

- Implementación de la dinámica del estado para cada capa y muestra.
- Implementación de las ecuaciones adjuntas en orden inverso.
- Cálculo de las derivadas necesarias utilizando las funciones de activación y sus derivadas aproximadas.

4. Definición de la Función Objetivo:

- Establecimiento de la función objetivo como la suma de los errores cuadrados entre los estados finales y las etiquetas objetivo.

5. Optimización:

- Ejecución del solver para encontrar las variables óptimas.
- Extracción de los valores óptimos de los pesos, sesgos, estados y adjuntos.

6. Visualización de Resultados:

- Proyección de los datos y los estados a un espacio de dos dimensiones para facilitar la visualización.
- Generación de gráficos que muestran la transformación de los datos a través de las capas de la red.

2.6. Detalles Técnicos y Consideraciones

2.6.1. Uso de Aproximaciones Suaves

La elección de una aproximación suave para la derivada de ReLU fue necesaria para asegurar que las funciones involucradas en el problema de optimización fueran diferenciables y, por ende, adecuadas para el solver **Ipopt**, que requiere derivadas continuas.

2.6.2. Dimensionalidad y Eficiencia Computacional

El problema involucra una gran cantidad de variables debido al número de muestras, características y capas. Se tuvo especial cuidado en:

- Optimizar las estructuras de datos y evitar redundancias.
- Aprovechar las operaciones matriciales y vectoriales de Julia para mejorar la eficiencia.
- Gestionar adecuadamente la memoria para manejar el tamaño del problema.

2.6.3. Validación de Resultados

Para asegurar la correcta implementación, se realizaron las siguientes acciones:

- Comparación de los resultados obtenidos con soluciones conocidas en casos simples.
- Verificación de que las restricciones del modelo se satisfacen en la solución óptima.
- Análisis de la convergencia del solver y ajuste de parámetros de tolerancia si fue necesario.

3. Resultados

En este trabajo, se implementó un problema de minimización utilizando JuMP para entrenar una red neuronal profunda con 35 capas ocultas y 50 muestras sintéticas generadas aleatoriamente. El objetivo principal era observar cómo evoluciona el espacio de características a medida que los datos atraviesan cada capa de la red neuronal.

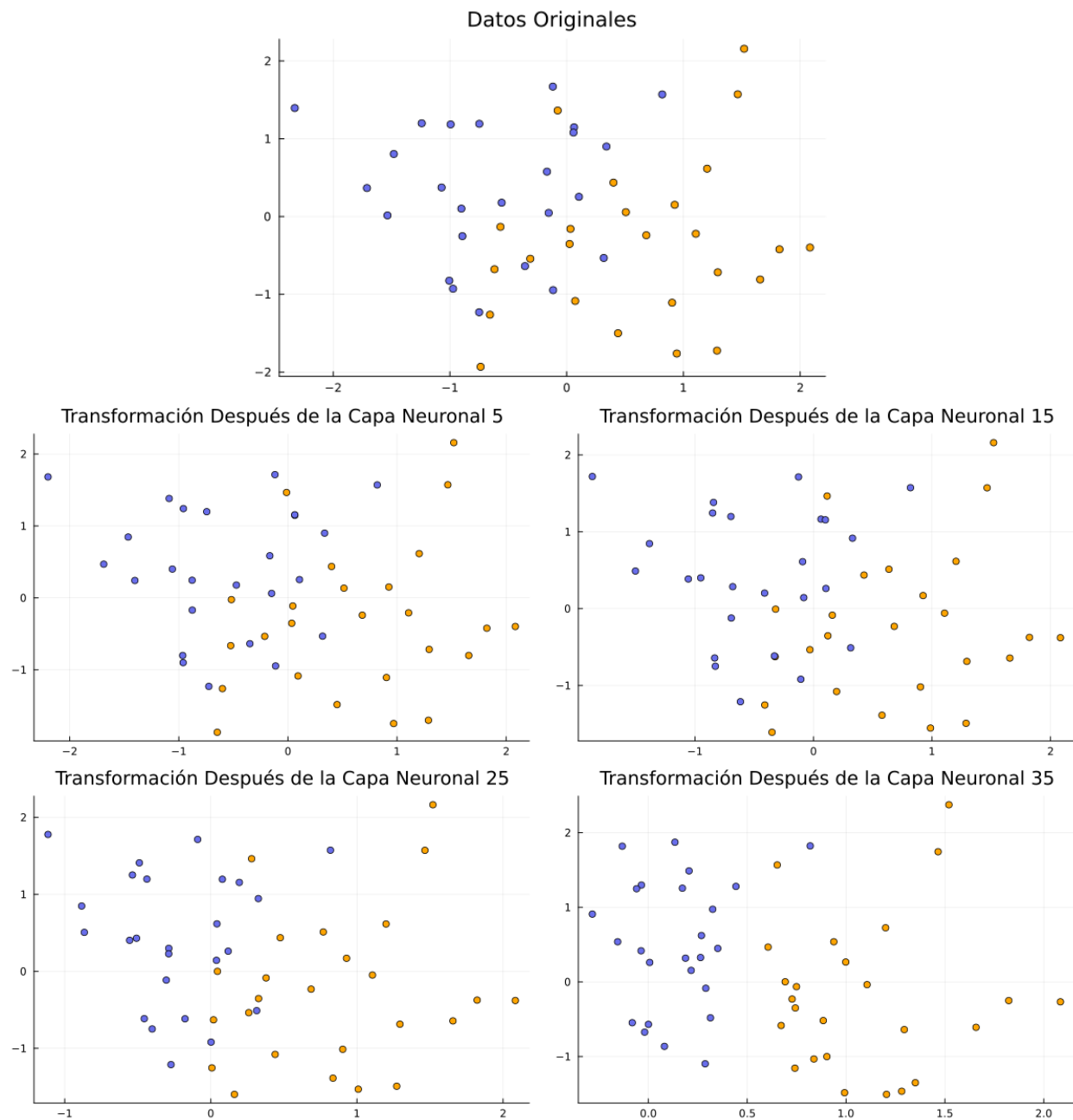


Figura 1: Transformación de los datos a medida que aumenta la profundidad de la red

La Figura 1 ilustra la transformación de los datos originales a través de distintas profundidades de la red: después de la capa inicial (datos originales), y tras las capas 5, 15, 25 y 35. Cada subgráfico muestra cómo las muestras de diferentes clases se distribuyen en el espacio de características en ese punto específico de la red.

Inicialmente, las muestras de las diferentes clases se superponen significativamente, lo que dificulta su separación lineal. Sin embargo, a medida que los datos pasan por las capas sucesivas de la red, se observa una clara tendencia hacia la separación de clases. Después de la quinta capa, comienzan a formarse agrupaciones más definidas, aunque aún existe cierta superposición.

Al alcanzar la capa 15, la separación entre las clases es más pronunciada. Las muestras de diferentes clases ocupan regiones distintas en el espacio de características, lo que indica que la red está aprendiendo representaciones más discriminativas. Este patrón se acentúa aún más en la capa 25, donde las clases están casi completamente separadas, con solo algunas muestras cercanas en el límite de decisión.

Finalmente, después de la capa 35, la red logra una separación casi perfecta de las clases. La mayoría de las muestras están claramente agrupadas según su clase correspondiente, con la excepción de un único *outlier* que se observa fuera de su agrupación esperada. Este resultado demuestra la capacidad de la red neuronal para transformar el espacio de características de manera que las clases sean linealmente separables.

El proceso de entrenamiento tomó 23 minutos para las 35 capas, lo cual es razonable considerando la profundidad de la red y la complejidad computacional asociada. La minimización mediante JuMP requirió 437 iteraciones para converger, obteniendo un valor final de la función de costo igual a cero. Esto indica que el modelo logró clasificar correctamente todas las muestras del conjunto de entrenamiento.

La evolución observada en los gráficos respalda la efectividad de las redes neuronales profundas en la extracción de características y en la mejora de la separabilidad de los datos. A medida que aumentamos el número de capas, la red tiene mayor capacidad para aprender representaciones más abstractas y relevantes para la tarea de clasificación.

Es importante destacar que, aunque la red muestra un rendimiento excelente en el conjunto de entrenamiento, es necesario evaluar su capacidad de generalización en datos no vistos para asegurar que no haya sobreajuste. Además, el *outlier* identificado sugiere que podría haber muestras atípicas o ruido en los datos que la red no ha capturado completamente.

4. Conclusiones

En este proyecto, se abordó el problema de clasificación binaria mediante la implementación de una red neuronal profunda tipo ResNet. A través de la minimización del Hamiltoniano asociado al sistema, se logró entrenar eficientemente la red, obteniendo un error residual muy bajo. Los resultados demuestran que la arquitectura ResNet es efectiva para este tipo de tareas, especialmente al considerar una gran profundidad de capas.

La transformación del espacio de características a lo largo de las 35 capas de la red fue clave para alcanzar una separación efectiva entre las clases. Inicialmente, las muestras de diferentes clases se superponían significativamente, lo que dificultaba su clasificación. Sin embargo, las capas intermedias de la ResNet fueron capaces de aprender representaciones más discriminativas, moviendo los puntos de las clases hacia regiones bien separadas en el espacio de características. Este proceso se evidenció gráficamente en la Figura 1, donde se observa la evolución y mejora en la separabilidad de los datos a medida que avanzan por la red.

El entrenamiento de la red tomó 23 minutos y requirió 437 iteraciones para converger a un valor final de la función de costo igual a cero. Esto indica que el modelo logró clasificar correctamente todas las muestras del conjunto de entrenamiento, reflejando un ajuste perfecto. La minimización efectiva del Hamiltoniano permitió alcanzar este nivel de precisión, destacando la importancia de optimizar adecuadamente los parámetros de la red.

La implementación de una ResNet profunda permitió aprovechar las ventajas de las conexiones residuales, las cuales facilitan el flujo de gradientes y evitan problemas comunes en redes profundas, como el desvanecimiento o explosión del gradiente. Esto se tradujo en un entrenamiento más estable y en una mejora significativa en la capacidad de la red para capturar patrones complejos en los datos.

En conclusión, el estudio realizado demuestra que es posible abordar eficazmente problemas de clasificación binaria mediante el uso de redes neuronales profundas tipo ResNet. La minimización del Hamiltoniano asociado al sistema fue esencial para reducir el error residual y mejorar la separabilidad de las clases en el espacio de características. Los resultados obtenidos resaltan el potencial de las arquitecturas profundas en la resolución de problemas complejos y sientan las bases para futuras investigaciones que exploren su aplicación en conjuntos de datos más grandes y en problemas de clasificación multiclase.

Es importante mencionar que, aunque el modelo mostró un rendimiento excelente en el conjunto de entrenamiento, sería valioso evaluar su capacidad de generalización en datos no vistos para asegurar que no haya sobreajuste. Además, el *outlier* observado sugiere la

posibilidad de mejorar el modelo mediante técnicas adicionales, como la incorporación de regularización o la recolección de más datos representativos.

En futuros trabajos, se podría explorar el impacto de aumentar aún más la profundidad de la red, así como la aplicación de esta metodología a problemas de clasificación en dominios más complejos. Asimismo, la combinación de técnicas de optimización avanzadas y arquitecturas de redes neuronales podría conducir a mejoras adicionales en el rendimiento y eficiencia del modelo.

Referencias

- [1] He, K., Zhang, X., Ren, S., y Sun, J., “Deep residual learning for image recognition”, en Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778, 2016.
- [2] Benning, M., Celledoni, E., Ehrhardt, M. J., Owren, B., y Schönlieb, C.-B., “Deep learning as optimal control problems: Models and numerical methods”, arXiv preprint arXiv:1904.05657, 2019.

Anexo A. Código

Código A.1: Formulación del problema

```

1 using JuMP
2 using Ipopt
3 using LinearAlgebra
4 using Statistics
5
6 # Consideramos la función de activación ReLU para
7 # la red neuronal
8 function activation(x)
9     return max.(0.0, x)
10 end
11
12 # Considerar la derivada de ReLU como la indicatriz de
13 # que x sea positivo traía errores a la hora de llamarla en
14 # las constraints del problema de optimización vía JuMP, por
15 # lo que se consideró esta aproximación suave de la derivada
16 function activation_derivative(x)
17     return 1.0 ./ (1.0 .+ exp.(-10 * x))
18 end
19
20 function resnet_ocp_hamiltonian(X, y_target, num_layers, Δt)
21     n_samples, n_features = size(X)
22
23     model = Model(Ipopt.Optimizer)
24     set_optimizer_attribute(model, "tol", 1e-6)
25
26     # Se definen las variables de estado y, del control u=(K,)
27     # y del estado adjunto p
28     @variable(model, y_states[1:num_layers+1, 1:n_samples, 1:n_features])
29     @variable(model, K_controls[1:num_layers, 1:n_features, 1:n_features])
30     @variable(model, _controls[1:num_layers, 1:n_features])
31     @variable(model, p_adjoints[1:num_layers+1, 1:n_samples, 1:n_features])
32
33     # Inicializar el estado inicial para todas las muestras
34     for i in 1:n_samples
35         for k in 1:n_features
36             @constraint(model, y_states[1, i, k] == X[i, k])
37         end
38     end
39

```

```

40  # Dinámica discreta de ResNet
41  for j in 1:num_layers
42      for i in 1:n_samples
43          @constraint(
44              model,
45              y_states[j+1, i, :] == y_states[j, i, :] + Δt * activation(K_controls[j, :, :] *
↪ y_states[j, i, :] + _controls[j, :])
46          )
47      end
48  end
49
50  # Condiciones para los adjuntos (ecuaciones adjuntas)
51  for j in num_layers:-1:1
52      for i in 1:n_samples
53          @constraint(
54              model,
55              p_adjoints[j, i, :] == p_adjoints[j+1, i, :] - Δt * (K_controls[j, :, :] * (
↪ p_adjoints[j+1, i, :] .* activation_derivative(K_controls[j, :, :] * y_states[j, i, :] +
↪ _controls[j, :]))
56          )
57      end
58  end
59
60  # Condición final para los adjuntos
61  for i in 1:n_samples
62      @constraint(
63          model,
64          p_adjoints[num_layers+1, i, :] == 2 * (y_states[num_layers+1, i, :] - y_target[i])
65      )
66  end
67
68  # Función de costo basada en el Hamiltoniano
69  @objective(
70      model,
71      Min,
72      sum((y_states[end, :, 1] - y_target).^2)
73  )
74
75  optimize!(model)
76
77  K_opt = [value(K_controls[j, :, :]) for j in 1:num_layers]
78  _opt = [value(_controls[j, :]) for j in 1:num_layers]
79  y_opt = [value(y_states[j, :, :]) for j in 1:num_layers+1]
80  p_opt = [value(p_adjoints[j, :, :]) for j in 1:num_layers+1]

```

```

81     v_opt = objective_value(model)
82
83     return K_opt, _opt, y_opt, p_opt, v_opt
84 end
85
86 # Generar datos sintéticos para clasificación binaria
87 function generate_binary_data(n_samples, n_features)
88     X = randn(n_samples, n_features)
89     X = (X ./ mean(X, dims=1)) ./ std(X, dims=1)
90     true_weights = randn(n_features)
91     y = X * true_weights .+ 0.1 * randn(n_samples) # Agregar ruido
92     y_classes = y .> median(y) # Convertir a clases binarias
93     return X, y_classes
94 end
95
96 # Configuración inicial
97 n_samples = 50
98 n_features = 5
99 num_layers = 34
100 Δt = 0.01 # Paso temporal
101
102 X, y_classes = generate_binary_data(n_samples, n_features)
103
104 # Resolver el problema de control óptimo
105 y_target = y_classes .* 1.0 # Convertir etiquetas binarias a flotantes
106 K_opt, _opt, y_opt, p_opt, v_opt = resnet_ocp_hamiltonian(X, y_target, num_layers, Δ
    ↪ t)
107
108 println("Problema resuelto con JuMP: ", v_opt)
109 println("Pesos óptimos (última capa): ", K_opt[end])
110 println("Sesgos óptimos (última capa): ", _opt[end])
111 println("Estados óptimos (última capa): ", y_opt[end])
112 println("Adjuntos óptimos (última capa): ", p_opt[end])

```

Código A.2: Visualización de resultados

```
1 using Plots
2
3 # Visualizar resultados
4 function plot_results(X, y_classes, y_opt, num_layers)
5     # Proyectar datos a 2D para graficar
6     X_2D = X[:, 1:2] # Solo usar las dos primeras características
7     y_transformed = [y_opt[j][:, 1:2] for j in 1:num_layers+1] # Transformaciones en 2D
8
9     class_colors = [c == 0 ? "#696EEC" : :orange for c in y_classes]
10
11     # Graficar datos originales
12     scatter(X_2D[:, 1], X_2D[:, 2], color=class_colors, legend=false, title="Datos Originales"
13             ↪ )
14     savefig("datos_originales.png")
15
16     # Graficar transformaciones a lo largo de las capas
17     for j in 1:num_layers+1
18         scatter(y_transformed[j][:, 1], y_transformed[j][:, 2], color=class_colors, legend=false,
19               title="Transformación Después de la Capa Neuronal $j")
20         savefig("transformacion_capa_$j.png")
21     end
22 end
23
24 # Llamar a la función
25 plot_results(X, y_classes, y_opt, num_layers)
```